

MẠNG MÁY TÍNH

Chương 3

Tầng giao vận

Tài liệu mẫu để thử Socratic Exam

Nhóm Socratic Exam biên soạn tóm lược cho mục đích học tập và trình diễn
Hackathon CNTT 2026 · Khoa CNTT, Trường Đại học Xây dựng Hà Nội

Mục tiêu của chương

Sau khi học xong chương này, người học cần:

- Giải thích được vai trò của tầng giao vận và sự khác nhau giữa truyền thông giữa các máy với truyền thông giữa các tiến trình.
- Mô tả được cách dồn kênh và phân kênh dựa trên số hiệu cổng.
- So sánh được UDP với TCP và nêu được khi nào nên chọn mỗi giao thức.
- Trình bày được các cơ chế cơ bản của truyền dữ liệu tin cậy: checksum, báo nhận, số thứ tự, đồng hồ đếm ngược và đường ống.
- Giải thích được cách TCP thiết lập kết nối, ước lượng thời gian khứ hồi và kiểm soát luồng.
- Phân tích được hành vi của cửa sổ tắc nghẽn trong TCP Tahoe, TCP Reno và TCP CUBIC.

Nội dung chương

3.1 Dịch vụ của tầng giao vận

3.2 Dồn kênh và phân kênh

3.3 UDP: vận chuyển không kết nối

3.4 Nguyên lý truyền dữ liệu tin cậy

3.5 TCP: vận chuyển hướng kết nối

3.6 Kiểm soát luồng

3.7 Kiểm soát tắc nghẽn của TCP

Câu hỏi ôn tập

3.1 Dịch vụ của tầng giao vận

Tầng giao vận nằm giữa tầng ứng dụng và tầng mạng. Nhiệm vụ chính của nó là cung cấp truyền thông logic giữa các tiến trình ứng dụng chạy trên những máy khác nhau. “Logic” ở đây có nghĩa là, nhìn từ phía ứng dụng, hai tiến trình như được nối thẳng với nhau, dù thực tế dữ liệu đi qua nhiều bộ định tuyến và nhiều loại đường truyền.

Cần phân biệt tầng giao vận với tầng mạng. Tầng mạng cung cấp truyền thông logic giữa các máy, còn tầng giao vận mở rộng dịch vụ đó thành truyền thông giữa các tiến trình. Có thể hình dung hai gia đình ở hai thành phố gửi thư cho nhau: dịch vụ bưu chính đưa thư từ nhà này đến nhà kia (tầng mạng), còn một người trong mỗi nhà thu gom thư của cả nhà và phát thư đến đúng từng người (tầng giao vận). Người thu gom thư chỉ làm việc trong nhà và không quyết định thư đi đường nào giữa hai thành phố.

Giao thức tầng mạng của Internet là IP. IP cung cấp dịch vụ “nỗ lực tối đa” (best-effort): nó cố chuyển gói đến đích nhưng không cam kết gói sẽ đến, không cam kết đúng thứ tự và không cam kết dữ liệu còn nguyên vẹn. Vì vậy, mọi đảm bảo về độ tin cậy mà ứng dụng cần đều phải được xây dựng ở các tầng phía trên.

Ở bên gửi, tầng giao vận nhận thông điệp từ ứng dụng, chia thành các phần nhỏ nếu cần, gắn thêm tiêu đề để tạo thành đoạn (segment) rồi chuyển xuống tầng mạng. Ở bên nhận, tầng giao vận đọc tiêu đề, ghép dữ liệu và chuyển lên đúng tiến trình. Internet cung cấp cho ứng dụng hai giao thức tầng giao vận chính: UDP (User Datagram Protocol), không kết nối và không tin cậy; và TCP (Transmission Control Protocol), hướng kết nối và tin cậy.

3.2 Dồn kênh và phân kênh

Một máy tính thường chạy cùng lúc nhiều tiến trình mạng: trình duyệt, ứng dụng nhắn tin, dịch vụ đồng bộ tệp. Mỗi tiến trình trao đổi dữ liệu qua một hoặc nhiều socket. Khi một đoạn đến, tầng giao vận phải biết đoạn đó thuộc về socket nào.

Việc chuyển dữ liệu trong đoạn đến đúng socket gọi là phân kênh (demultiplexing). Công việc ngược lại ở bên gửi, tức gom dữ liệu từ nhiều socket, gắn tiêu đề rồi chuyển xuống tầng mạng, gọi là dồn kênh (multiplexing). Để làm được hai việc này, mỗi đoạn mang số hiệu cổng nguồn và số hiệu cổng đích. Số hiệu cổng dài 16 bit, nhận giá trị từ 0 đến 65535. Các cổng từ 0 đến 1023 là cổng phổ biến, dành cho những giao thức chuẩn như HTTP (cổng 80), HTTPS (cổng 443) và DNS (cổng 53).

Với UDP, một socket được xác định bởi cặp (địa chỉ IP đích, cổng đích). Hai đoạn đến từ hai máy khác nhau nhưng có cùng địa chỉ IP đích và cùng cổng đích sẽ được chuyển vào cùng một socket. Cổng nguồn trong đoạn UDP đóng vai trò “địa chỉ trả lời” để bên nhận gửi phản hồi.

Với TCP, một socket được xác định bởi bộ bốn (IP nguồn, cổng nguồn, IP đích, cổng đích). Nhờ vậy, một máy chủ web có thể giữ nhiều socket cùng dùng cổng 80, mỗi socket phục vụ một kết nối riêng. Hai đoạn TCP có cùng cổng đích nhưng khác IP nguồn hoặc khác cổng nguồn sẽ đi vào hai socket khác nhau.

3.3 UDP: vận chuyển không kết nối

UDP chỉ làm những việc tối thiểu mà một giao thức tầng giao vận phải làm: dồn kênh, phân kênh và kiểm tra lỗi đơn giản. Khi dùng UDP, ứng dụng gần như trao đổi trực tiếp với IP.

Dù không đảm bảo độ tin cậy, UDP vẫn phù hợp với nhiều ứng dụng vì những lý do sau:

- Không cần thiết lập kết nối, nên không mất thêm thời gian khứ hồi trước khi gửi dữ liệu. Đây là lý do DNS dùng UDP.
- Không giữ trạng thái kết nối, nên một máy chủ phục vụ được nhiều máy khách hơn.

- Tiêu đề chỉ dài 8 byte, nhỏ hơn nhiều so với 20 byte của TCP.
- Ứng dụng kiểm soát tốt hơn thời điểm gửi: dữ liệu được đẩy xuống mạng ngay, không bị cơ chế kiểm soát tắc nghẽn làm chậm lại.

Các ứng dụng truyền âm thanh và hình ảnh thời gian thực, trò chơi trực tuyến và giao thức QUIC (nền tảng của HTTP/3) đều chạy trên UDP. QUIC tự xây dựng độ tin cậy và kiểm soát tắc nghẽn ở tầng ứng dụng thay vì dựa vào TCP.

Tiêu đề UDP gồm bốn trường, mỗi trường 16 bit: cổng nguồn, cổng đích, độ dài (tính cả tiêu đề) và checksum.

Checksum dùng để phát hiện bit bị thay đổi trên đường đi. Bên gửi coi dữ liệu là dãy các từ 16 bit, cộng chúng lại theo số học bù một (bit tràn được cộng vòng vào bit thấp nhất), rồi lấy bù một của tổng làm checksum. Bên nhận cộng tất cả các từ, kể cả checksum. Nếu không có lỗi, kết quả là 16 bit đều bằng 1; chỉ cần một bit bằng 0 là biết đoạn đã bị hỏng. UDP chỉ phát hiện lỗi chứ không sửa lỗi: đoạn hỏng bị bỏ đi, hoặc được chuyển lên ứng dụng kèm cảnh báo.

Tiêu chí	UDP	TCP
Kết nối	Không cần thiết lập	Bắt tay ba bước trước khi gửi dữ liệu
Độ tin cậy	Không đảm bảo	Dữ liệu đến đủ, không lỗi và đúng thứ tự
Kích thước tiêu đề	8 byte	Tối thiểu 20 byte
Kiểm soát luồng và tắc nghẽn	Không có	Có
Ứng dụng tiêu biểu	DNS, gọi thoại, trò chơi trực tuyến, QUIC	Web, thư điện tử, truyền tệp

Bảng 3.1 · So sánh UDP và TCP

3.4 Nguyên lý truyền dữ liệu tin cậy

Kênh truyền bên dưới có thể làm hỏng bit và làm mất gói. Một giao thức truyền tin cậy phải che giấu những lỗi đó, để ứng dụng thấy một kênh không có lỗi. Các cơ chế cơ bản gồm:

- Checksum để phát hiện gói bị hỏng.
- Báo nhận (ACK) để bên nhận cho bên gửi biết gói đã đến an toàn.
- Số thứ tự để bên nhận phân biệt gói mới với gói được gửi lại.
- Đồng hồ đếm ngược để bên gửi gửi lại khi gói hoặc ACK bị mất.

Giao thức đơn giản nhất là dừng và chờ (stop-and-wait): bên gửi gửi một gói rồi chờ ACK mới gửi gói tiếp theo. Cách này đúng nhưng rất lãng phí. Xét đường truyền 1 Gbps, thời gian khứ hồi (RTT) 30 ms và gói dài 1.000 byte. Thời gian đẩy một gói lên đường truyền là $L/R = 8.000 \text{ bit} / 10^9 \text{ bit/s}$ chia cho 10^9 bit/s , tức 8 micro giây. Hiệu suất sử dụng đường truyền là $U = (L/R) / (RTT + L/R) = 0,008 / 30,008 \approx 0,00027$, nghĩa là đường truyền chỉ thực sự bận khoảng 0,027% thời gian.

Giải pháp là cơ chế đường ống (pipelining): cho phép bên gửi gửi nhiều gói khi chưa nhận được báo nhận. Nếu cho phép ba gói cùng ở trên đường, hiệu suất tăng gần gấp ba. Đường ống đòi hỏi dài số thứ tự lớn hơn và bộ đệm ở cả hai phía. Có hai cách xử lý lỗi chính.

Go-Back-N

Bên gửi được có tối đa N gói chưa được báo nhận. Bên nhận chỉ nhận gói đúng thứ tự và dùng ACK tích lũy: ACK cho gói n nghĩa là mọi gói đến n đều đã nhận. Khi đồng hồ của gói cũ nhất hết giờ, bên gửi gửi lại gói đó và toàn bộ các gói sau nó. Cách này đơn giản, nhưng một gói mất có thể kéo theo việc gửi lại nhiều gói không cần thiết.

Lặp lại có chọn lọc

Với cơ chế lặp lại có chọn lọc (Selective Repeat), bên nhận báo nhận riêng từng gói và giữ lại các gói đến không đúng thứ tự trong bộ đệm. Bên gửi chỉ gửi lại những gói bị nghi là mất, mỗi gói có đồng hồ riêng. Để bên nhận không nhầm gói mới với gói cũ, kích thước cửa sổ không được lớn hơn một nửa không gian số thứ tự.

3.5 TCP: vận chuyển hướng kết nối

TCP là giao thức điểm nối điểm, song công toàn phần (dữ liệu đi được theo cả hai chiều cùng lúc) và cung cấp một luồng byte tin cậy, đúng thứ tự. Trước khi trao đổi dữ liệu, hai bên phải thiết lập kết nối và khởi tạo trạng thái, gồm số thứ tự, bộ đệm và các biến điều khiển. Trạng thái này chỉ nằm ở hai máy đầu cuối; các bộ định tuyến ở giữa không biết gì về kết nối.

Lượng dữ liệu ứng dụng tối đa trong một đoạn gọi là MSS (maximum segment size). Trên mạng Ethernet có MTU 1.500 byte, MSS thường là 1.460 byte, vì còn 20 byte tiêu đề IP và 20 byte tiêu đề TCP.

Cấu trúc đoạn TCP

Tiêu đề TCP gồm cổng nguồn và cổng đích (16 bit mỗi trường), số thứ tự và số báo nhận (32 bit mỗi trường), độ dài tiêu đề, các cờ điều khiển (SYN, ACK, FIN, RST, PSH, URG), cửa sổ nhận (16 bit), checksum, con trỏ khẩn và phần tùy chọn. Khi không có tùy chọn, tiêu đề dài 20 byte.

Số thứ tự và số báo nhận

TCP đánh số theo byte trong luồng dữ liệu chứ không theo đoạn. Số thứ tự của một đoạn là số thứ tự của byte đầu tiên trong đoạn. Số báo nhận là số thứ tự của byte tiếp theo mà bên nhận đang chờ. Ví dụ, nếu bên nhận đã có đủ các byte từ 0 đến 535, nó gửi số báo nhận 536. TCP dùng số báo nhận tích lũy: một ACK xác nhận mọi byte đứng trước số báo nhận.

Ước lượng thời gian khứ hồi

TCP phải đặt thời gian chờ lớn hơn RTT, nhưng không quá lớn để còn kịp phản ứng khi gói bị mất. Mỗi lần đo được một mẫu $SampleRTT$, TCP cập nhật:

$$EstimatedRTT = (1 - \alpha) \cdot EstimatedRTT + \alpha \cdot SampleRTT, \text{ với } \alpha = 0,125$$

$$DevRTT = (1 - \beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|, \text{ với } \beta = 0,25$$

$$TimeoutInterval = EstimatedRTT + 4 \cdot DevRTT$$

TCP không đo $SampleRTT$ cho các đoạn đã gửi lại, vì không biết ACK nhận được ứng với lần gửi nào. Mỗi lần hết giờ, TCP gửi lại đoạn chưa được báo nhận và nhân đôi $TimeoutInterval$, để tránh dồn thêm gói vào một mạng có thể đang tắc nghẽn.

Thiết lập kết nối: bắt tay ba bước

1. Máy khách gửi đoạn SYN (cờ SYN bằng 1), không chứa dữ liệu, mang số thứ tự ban đầu $client_isn$ được chọn ngẫu nhiên.
2. Máy chủ cấp bộ đệm và biến cho kết nối, rồi trả đoạn SYNACK với số thứ tự ban đầu $server_isn$ và số báo nhận $client_isn + 1$.
3. Máy khách cấp bộ đệm và biến, rồi gửi ACK với số báo nhận $server_isn + 1$. Đoạn thứ ba này đã có thể mang dữ liệu.

Đóng kết nối

Mỗi bên gửi một đoạn FIN và chờ bên kia báo nhận. Bên chủ động đóng ở trạng thái `TIME_WAIT` một khoảng thời gian, thường từ 30 giây đến 2 phút, trước khi giải phóng hoàn toàn, để còn gửi lại được ACK cuối nếu ACK đó bị mất.

3.6 Kiểm soát luồng

Mỗi bên của kết nối TCP có một bộ đệm nhận. Nếu ứng dụng đọc dữ liệu chậm hơn tốc độ dữ liệu đến, bộ đệm sẽ tràn. Kiểm soát luồng (flow control) giúp bên gửi không làm tràn bộ đệm của bên nhận.

Bên nhận quảng bá cửa sổ nhận $rwnd$, là phần còn trống của bộ đệm:

$$rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$$

Bên gửi giữ lượng dữ liệu đã gửi mà chưa được báo nhận không vượt quá $rwnd$:

$$LastByteSent - LastByteAcked \leq rwnd$$

Khi $rwnd$ bằng 0, bên gửi vẫn tiếp tục gửi các đoạn thăm dò chứa 1 byte dữ liệu. Nhờ đó, khi ứng dụng đọc bớt dữ liệu và bộ đệm có chỗ trống, bên nhận có dịp báo cho bên gửi giá trị $rwnd$ mới.

Cần phân biệt kiểm soát luồng với kiểm soát tắc nghẽn. Kiểm soát luồng bảo vệ bên nhận khỏi bị gửi quá nhanh; kiểm soát tắc nghẽn bảo vệ mạng khỏi bị quá tải. Hai cơ chế cùng hạn chế tốc độ gửi nhưng vì hai lý do khác nhau.

3.7 Kiểm soát tắc nghẽn của TCP

Tắc nghẽn xảy ra khi quá nhiều nguồn gửi dữ liệu quá nhanh so với khả năng của mạng. Hàng đợi tại bộ định tuyến đầy lên, độ trễ tăng, gói bị bỏ và bên gửi phải gửi lại, khiến băng thông bị dùng cho cả những bản sao không cần thiết.

TCP dùng kiểm soát tắc nghẽn đầu cuối: mạng không báo trực tiếp về tắc nghẽn, bên gửi tự suy ra từ các dấu hiệu mất gói, gồm hết giờ và ba ACK trùng lặp. Bên gửi duy trì biến cửa sổ tắc nghẽn $cwnd$ và giới hạn lượng dữ liệu chưa được báo nhận:

$$LastByteSent - LastByteAcked \leq \min(cwnd, rwnd)$$

Vì vậy, tốc độ gửi xấp xỉ $cwnd / RTT$ byte mỗi giây.

3.7.1 Khởi động chậm và tránh tắc nghẽn

Khi kết nối bắt đầu, $cwnd$ thường được đặt bằng 1 MSS. Trong pha khởi động chậm (slow start), $cwnd$ tăng thêm 1 MSS cho mỗi ACK nhận được, nên $cwnd$ tăng gấp đôi sau mỗi RTT. Tên gọi “chậm” chỉ nói về điểm xuất phát; tốc độ tăng thực ra theo hàm mũ.

Khởi động chậm dừng khi $cwnd$ đạt ngưỡng $ssthresh$; khi đó TCP chuyển sang pha tránh tắc nghẽn (congestion avoidance). Trong pha này, $cwnd$ chỉ tăng khoảng 1 MSS sau mỗi RTT, tức tăng tuyến tính. Khi phát hiện mất gói, TCP giảm $cwnd$ đi một nửa. Cách điều chỉnh “tăng cộng, giảm nhân” này gọi là AIMD (additive increase, multiplicative decrease) và làm cho đồ thị $cwnd$ theo thời gian có dạng răng cưa.

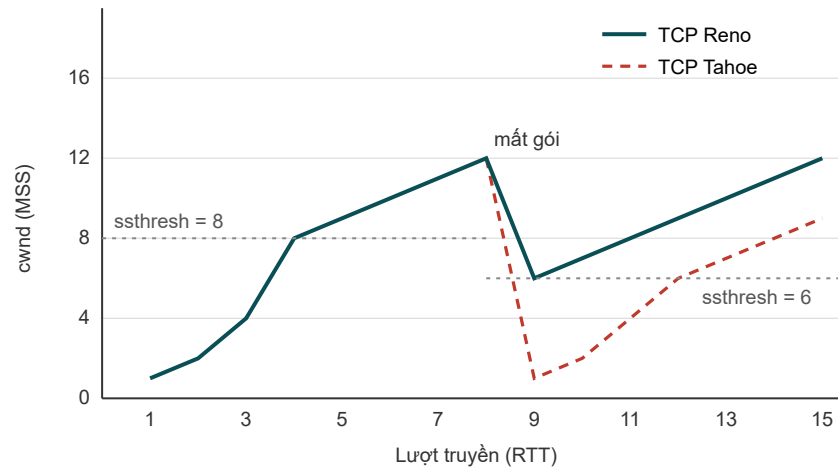
Khi xảy ra hết giờ, cả TCP Tahoe và TCP Reno đều đặt $ssthresh = cwnd/2$, đặt $cwnd$ về 1 MSS và bắt đầu lại pha khởi động chậm. Hết giờ được coi là dấu hiệu tắc nghẽn nghiêm trọng, vì ngay cả các ACK cũng không về được đến bên gửi.

3.7.2 Truyền lại nhanh và phục hồi nhanh

Khi bên gửi nhận được ba ACK trùng lặp cho cùng một đoạn, TCP coi đó là dấu hiệu một gói đã bị mất nhưng mạng vẫn còn chuyển được dữ liệu. Thay vì chờ hết thời gian RTO, bên gửi truyền lại ngay đoạn bị thiếu. Cơ chế này gọi là truyền lại nhanh (Fast Retransmit).

Sau đó TCP Reno đặt $ssthresh = cwnd/2$ và $cwnd = ssthresh + 3 \text{ MSS}$, rồi chuyển sang pha phục hồi nhanh (Fast Recovery). Trong pha này, mỗi ACK trùng lặp nhận thêm làm $cwnd$ tăng 1 MSS. Khi ACK mới đến, $cwnd$ được đặt lại bằng $ssthresh$ và TCP quay về pha tránh tắc nghẽn.

TCP Tahoe thì khác: dù phát hiện mất gói bằng timeout hay bằng ba ACK trùng lặp, Tahoe đều đặt $cwnd$ về 1 MSS và bắt đầu lại từ pha khởi động chậm.



Hình 3.1 · $cwnd$ theo thời gian khi mất gói ở lượt 8 (ba ACK trùng lặp). Nét liền: TCP Reno, nét đứt: TCP Tahoe.

3.7.3 Tính công bằng

Giả sử K kết nối TCP cùng đi qua một liên kết nút cổ chai có tốc độ R . Cơ chế kiểm soát tắc nghẽn được gọi là công bằng nếu mỗi kết nối nhận được tốc độ trung bình xấp xỉ R/K . Với hai kết nối có cùng MSS và cùng RTT, AIMD làm phân băng thông của hai kết nối dần tiến về đường chia đều: khi cùng tăng, hai kết nối tăng cùng một lượng; khi cùng giảm, kết nối đang chiếm nhiều hơn sẽ mất nhiều hơn.

Trong thực tế, kết nối có RTT nhỏ hơn tăng $cwnd$ nhanh hơn nên thường chiếm nhiều băng thông hơn. Ngoài ra, một ứng dụng mở nhiều kết nối TCP song song, như một số trình duyệt và công cụ tải tệp, sẽ nhận được phần lớn hơn băng thông của liên kết nghẽn.

3.7.4 TCP CUBIC

TCP CUBIC là thuật toán mặc định trên Linux và nhiều hệ điều hành hiện đại. Ý tưởng của CUBIC là: ngay sau khi mất gói, tình trạng của mạng có lẽ chưa thay đổi nhiều, nên cần nhanh chóng quay lại gần mức cửa sổ W_{max} lúc xảy ra mất gói, rồi mới thận trọng thăm dò vượt lên. CUBIC tăng $cwnd$ theo hàm bậc ba của thời gian tính từ lần mất gói gần nhất: tăng nhanh khi còn xa W_{max} , chậm lại khi đến gần W_{max} , rồi tăng nhanh dần sau khi vượt qua.

Vì tăng theo thời gian thay vì theo số ACK nhận được, CUBIC tận dụng tốt các đường truyền băng thông lớn và độ trễ cao, đồng thời ít bất lợi hơn cho các kết nối có RTT lớn.

Câu hỏi ôn tập

1. Vì sao nói tầng giao vận cung cấp truyền thông logic giữa các tiến trình, còn tầng mạng cung cấp truyền thông logic giữa các máy?
2. Hai đoạn UDP đến từ hai máy khác nhau, có cùng địa chỉ IP đích và cùng cổng đích, sẽ được chuyển vào bao nhiêu socket? Nếu là TCP thì sao?
3. Nêu ba lý do khiến một ứng dụng chọn UDP thay vì TCP.
4. Tính hiệu suất của giao thức dừng và chờ với đường truyền 100 Mbps, RTT 20 ms và gói dài 1.500 byte.
5. So sánh Go-Back-N và lặp lại có chọn lọc khi một gói ở giữa cửa sổ bị mất.
6. Vì sao TCP không đo SampleRTT cho đoạn đã gửi lại?
7. Phân biệt kiểm soát luồng và kiểm soát tắc nghẽn. Mỗi cơ chế dùng biến nào?
8. Giải thích vì sao ba ACK trùng lặp được xem là dấu hiệu tắc nghẽn nhẹ hơn so với hết giờ.
9. Khi bên gửi nhận được ba ACK trùng lặp, cwnd đang là 12 MSS. Hãy cho biết cwnd và ssthresh ngay sau đó với TCP Tahoe và với TCP Reno.
10. CUBIC khác AIMD truyền thống ở điểm nào, và vì sao điều đó có lợi trên đường truyền băng thông lớn, độ trễ cao?